Yancheng (Jason) Liu Research Summary

MRes in AI and Machine Learning, Imperial College London
j.liu26@imperial.ac.uk | setsuna113.github.io | github.com/setsuna113

My research is on memory for long-horizon LLM agents: what the KV cache and other inference state should keep, in what form and where, when each decision's cost appears only turns later. I work on both the methods that make these decisions and the serving systems that run them.

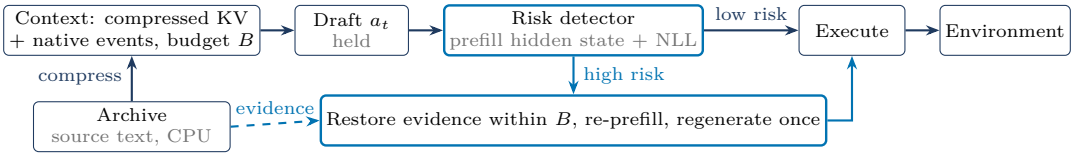
Racer: Recoverable KV Cache Management for Multi-Turn LLM Agents

Aug–Sep 2026

Co-first author, under review at ICLR 2027 · SJTU, Prof. Shengzhong Liu

Problem. Agent prompts grow with every tool response; on BFCL Long Context the full history is 21.45× a 256-token history budget. Unlike compaction into a summary, KV compression adds no decode and keeps the source restorable. But in a closed loop a wrong action changes the environment and every later observation: six KV compressors all fall far below full context (mean task success 10.5% vs 32.5%), because a compressor decides what to keep without seeing the outcome.

Method. Racer checks each action at the draft, before it executes. A linear detector reads the hidden state prefill already computed plus the draft's average NLL, with a counterfactual label: does the turn fail if this draft commits without recovery? Risky drafts (18–22% of decisions) are regenerated once after restoring archived evidence within the same budget.



System. Recovery edits the middle of the history, but SGLang's RadixCache only reuses an exact, unchanging prefix. Compressed blocks are therefore position-independent (C^2KV): keys cached before RoPE, keyed by source token IDs, rotated at assembly, held in a GPU-resident LRU pool. New history is compressed on a side CUDA stream during decode, and the engine returns the hidden state the detector reads, so the check adds no forward pass.

Results. Across six KV-compression backends, mean task success on BFCL Long Context rose from 10.5% to 15.1% (43.7% relative), and all six reached 14–16%, so the backend can be chosen for systems reasons. On one SGLang engine (Qwen3-4B, RTX 4090) at 16 concurrent tasks, C^2KV + Racer completed 1.67× the tasks per hour of full-context execution and cut p95 latency from 725 s to 348 s.

My part. The risk detector and the evidence-recovery step, training the C^2KV compressor on correct tool-call continuations, and the concurrent SGLang serving runtime with session-owned compressed KV.

ShapeFlow: Serving Agentic Workflows with Late-Bound Representations

May–Jul 2026

SJTU, Prof. Shengzhong Liu · github.com/setsuna113/shapeflow-p1-study

Problem. A few deep-research tasks saturate a local GPU pool. On Open Deep Research with Qwen3-14B, decoding a summary of every retrieved page took 88% of LLM time; prefill was about 2%. Saturated throughput is decode rate over tokens decoded per task, so scheduling cannot raise it (three policies gave the same 84.7 tasks/h); only the form of the evidence can. **Design.** At each evidence handoff the agent offers two equivalent plans: per-page summaries, or a decision model that outputs only span IDs, with the text kept in CPU memory until the report is written. A scheduler I built beside an unmodified vLLM reads engine load every tick and picks the plan together with request admission. **Results.** On BrowseComp-Plus (pre-registered, 80 tasks, 7 arms), CPU-selected spans in place of per-page summaries cut vLLM busy time per task by 67%; the scheduler drove live agents end to end.

Mechanistic Interventions and Failure Signals in Multimodal Models

Jun–Sep 2026

Cambridge Machine Intelligence Lab, Dr Mengjie Qian and Prof. Kate Knill · manuscript in preparation for Interspeech 2027

With a multimodal Transformer classifier's predictions and errors fixed, MC-dropout mutual information raised error-detection AUROC by 0.07–0.12 under three training objectives. I proved that it measures dropout-induced margin fluctuation and traced it by intervention: perturbing only the lower half of the blocks matched all layers; the upper half gave almost no gain.

Coupled Muon: Product-Space Orthogonalization

Mar–May 2026

SJTU AI, Yiting Chen and Prof. Junchi Yan · github.com/setsuna113/coupled-muon-nanogpt

Muon orthogonalizes each matrix's update separately, leaving the Q–K and V–O product updates uncontrolled; I designed Coupled Muon to orthogonalize the product's update. Across 960 runs on a 60M-parameter ladder it lowered validation loss by 0.007–0.024 nats relative to Muon on every RoPE dense and MoE variant.

Now: Inference State in Hybrid Models

Sep 2026–

MRes, Imperial College London, with Prof. Peter Pietzuch

Hybrid and sparse-attention models hold several kinds of state (attention KV, sliding-window KV, linear-attention state, sparse payload) that differ in size and access. While agents wait on tools, I study which state stays in GPU memory, what moves to CPU memory and when it comes back, targeting SLO goodput on SGLang and vLLM.